

Architecting Philon: A Decision-Ready Blueprint for High-Fidelity, Local-First Document Conversion

Executive Recommendation: The Philon Thesis

The central thesis of Philon is to evolve beyond the single-pipeline, LLM-reliant model of its predecessor, Marker, into a sophisticated, multi-stage, adaptive routing system. This new architecture will prioritize both accuracy and performance by intelligently decoupling extraction from rendering and applying computational resources only where necessary. Philon's core strategy is "native-first, AI-second": it will leverage the inherent semantic structure of born-digital PDFs whenever possible, using fast, low-level parsers to extract high-quality text with minimal latency. High-resolution rasterization, Optical Character Recognition (OCR), and Vision-Language Model (VLM) repair mechanisms will be deployed as targeted, intelligent fallbacks for regions of uncertainty, complex layouts, or specialized content like mathematical formulae. This approach directly addresses the primary performance bottlenecks of Marker, which relies heavily on computationally expensive OCR for nearly all pages ^{73 74}, while enabling continued accuracy improvements through selective AI usage.

To enable this advanced architecture and ensure broad adoption, Philon must be built on a permissive software license (e.g., MIT or Apache 2.0). This decision is non-negotiable, as it allows for integration into commercial applications and fosters a healthy ecosystem of plugins and extensions, distinguishing it from Marker's GPL license which can complicate proprietary use ^{70 72}. The platform's target users are developers, researchers, and enterprises working in data science, information retrieval, and AI-driven workflows who require high-fidelity document conversion for tasks such as RAG pipelines, data ingestion, and accessibility remediation ^{14 55}. Philon positions itself not just as another converter, but as a foundational tool for turning unstructured documents into reliable, machine-readable knowledge ²⁷.

The guiding principles for Philon's development are: 1. **Source Grounding and Provenance:** Every piece of extracted information must be traceable back to its origin in the source document, including page number, coordinates, extraction method, and

confidence score. This transparency builds trust and enables debugging. 2. **Determinism in Local Mode:** In its default local-only mode, Philon must produce bit-for-bit reproducible outputs for the same input file, ensuring reliability and consistency across runs. 3. **Selective AI Usage:** While acknowledging the power of modern AI, Philon will not rely on it as the primary extraction mechanism. Instead, AI models will serve as precision instruments for verification, correction, and handling edge cases, used judiciously to balance cost, latency, and accuracy. 4. **Performance First:** The adaptive routing architecture is designed to be exceptionally fast for common, well-structured documents, outperforming Marker in these scenarios while retaining the capability to handle complexity when required. 5. **Public Accountability:** Philon will be backed by a rigorous, public, and reproducible benchmarking framework designed to prevent gaming and provide verifiable claims of superiority over competing solutions.

By adhering to this thesis, Philon will deliver a next-generation document conversion platform that is more accurate, faster, flexible, and commercially viable than any solution currently available.

Marker Codebase Audit and Competitive Landscape

A thorough audit of the Marker codebase and its surrounding ecosystem reveals its strengths, critical limitations, and the strategic opportunities for Philon. This section provides an evidence-based inventory of Marker's capabilities and contrasts them with the broader competitive landscape across six key technical domains.

The Marker repository (`datalab-to/marker`) serves as the foundation for this analysis ³⁷. Its primary function is converting PDFs to Markdown, JSON, chunks, and HTML ³⁷. It operates via a CLI, API, and server interface, with a renderer architecture focused on extracting blocks of content ⁶⁹ ⁷⁷. However, its effectiveness is heavily dependent on external models and tools. It uses Surya for OCR and layout analysis ³², PyTorch for model inference, and LLM services for repair ⁷⁷. This reliance creates significant dependencies and performance bottlenecks. For instance, processing 94 diverse documents was reported to take over 60 minutes, indicating substantial latency ⁷³. Speed is further constrained by duplicated work, fixed-stages, and high VRAM requirements due to GPU-dependent model execution ⁷⁴. Licensing is a major constraint; the Marker code is under the GPL license, and its models use a modified OpenRAIL-style license, making direct reuse in a permissive/commercial architecture impossible without

significant reimplementation ⁷⁰. Despite these issues, Marker demonstrates state-of-the-art accuracy, particularly on complex documents like forms and scanned papers, achieving a 76.0% overall score on one benchmark and showing a 20-point lead over Docling on forms ^{37 102}.

Category	Marker Capabilities & Limitations	Evidence
Input/Output	PDF input; outputs Markdown, JSON, chunks, HTML. Limited to PDF format.	^{37 77}
Extraction Pipeline	Relies on rasterizing pages, running OCR (Surya), and using VLMs for repair. Fixed, sequential stages.	^{32 77}
Speed & Bottlenecks	High latency (e.g., >60 mins for 94 docs). GPU/CPU/MPS support exists but requires high VRAM.	^{73 74}
Accuracy Gaps	Does not convert 100% of equations to LaTeX. Accuracy degrades on some complex forms.	^{39 102}
Licensing	Codebase is GPL licensed. Models use a modified OpenRAIL-style license. Not suitable for permissive/commercial use.	⁷⁰

The competitive landscape is vibrant, with several strong contenders vying for dominance in the open-source space. Docling, developed by IBM, is a highly efficient parser that outputs structured JSON and has shown strong performance, particularly on table extraction (97.9% accuracy on one IBM benchmark) ^{5 85}. MinerU, also from Datalab, focuses on OCR upgrades and pipeline optimization, with recent versions improving OCR capabilities significantly ¹⁰⁷¹⁰⁸. Unstructured.io offers a comprehensive platform for turning various file types into structured JSON, positioning itself as a data ingestion layer for AI systems ^{21 23}. Other notable tools include MarkItDown, known for preserving structural elements ⁴², and Nougat, a specialized academic parser that understands LaTeX ⁸⁶.

Tool/ Framework	Key Strengths	Key Weaknesses	License
Marker	High accuracy on complex documents (forms, scans); good equation/table handling.	Very slow; high VRAM/GPU dependency; GPL license limits commercial use.	GPL / Modified OpenRAIL
Docling	Efficient; outputs structured JSON; strong table extraction accuracy.	May lag behind Marker on extreme complexity.	MIT / Apache 2.0 ²⁹
MinerU	Focus on OCR capability upgrades; optimized processing pipeline.	Accuracy can vary; less mature than Marker in some areas.	MIT / Apache 2.0 ²⁹
Unstructured.io	Broad file type support (>64); excellent for data ingestion into AI pipelines.	Can be less precise for pure PDF-to-Markdown conversion compared to specialists.	Apache 2.0 ²⁹
Nougat	State-of-the-art for academic PDFs; understands and converts LaTeX.	Specialized for academic papers; not a general-purpose converter.	CC BY-NC-SA 4.0
Mathpix	Commercial service with very high accuracy; excellent for math and tables.	Proprietary; not open-source; costs associated with use.	Proprietary

In the native PDF parsing domain, libraries like PyMuPDF and pdfminer are fundamental building blocks ^{11 103}. They excel at extracting text, fonts, links, and basic structure from born-digital PDFs without rasterization, offering a potential path to significant speed improvements for simple documents ¹¹. For OCR, the landscape has evolved rapidly. While Tesseract remains a baseline option, newer models like PaddleOCR-VL, OlmOCR-2, and MistralOCR offer superior accuracy, multilingual support, and better layout preservation ^{30 67 68}. Critically, many of these modern alternatives carry permissive licenses (Apache 2.0 or MIT), making them suitable candidates for Philon's architecture ^{28 29}.

Layout analysis is another critical area. Tools like LayoutParser, DeepDoctection, and MinerU provide robust frameworks for detecting document elements like tables, figures, and headings ⁶. Benchmarks like DocBank and ReadingBank provide standardized datasets for evaluating the performance of these models ^{40 41}. For tables and formulas, the challenge lies in reconstruction. OmniDocBench has emerged as a key benchmark for evaluating models on these specific tasks, highlighting the need for solutions that can accurately parse nested tables, borderless tables, and reconstruct LaTeX from images ^{87 89}. Finally, the role of LLMs and VLMs is shifting from being the primary extractor to being powerful repair agents. Their ability to perform cross-page reasoning and provide structured, constrained outputs makes them ideal for verifying and correcting errors made by other components in the pipeline ^{18 50}. This nuanced role is crucial for Philon's strategy of using AI selectively and effectively.

Proposed Philon Architecture: An Adaptive, Multi-Stage Processing Pipeline

To surpass Marker's performance and licensing constraints, Philon will implement an adaptive, multi-stage processing pipeline. This architecture moves away from Marker's monolithic, "one-size-fits-all" approach and instead employs a smart router to dynamically select the most appropriate processing tool for each region of a document based on real-time quality assessment. The core principle is to minimize unnecessary computation—specifically, the costly process of high-resolution rasterization and OCR—by leveraging native PDF structure wherever possible. This design directly targets the primary performance bottleneck of Marker, which applies its heavy OCR-based processing to every page regardless of its complexity or native readability ^{73 74}.

The proposed architecture is centered around a modular component design with clear plugin interfaces. At its heart is the **Router**, a decision-making engine that orchestrates the entire process. It receives a PDF file and delegates tasks to pluggable **Extractor Modules**. These modules are responsible for specific types of content: a `NativeTextExtractor`, an `OcrExtractor`, a `TableDetector`, a `FormulaConverter`, and so on. Once extraction is complete, the resulting data is assembled into the canonical **Document IR**. From there, pluggable **Renderer Modules** take the IR and produce the final desired output format, such as Markdown, Semantic HTML, or JSON. This decoupling of extraction from rendering is fundamental to achieving deterministic outputs and enabling future extensibility ².

The data flow begins when a PDF is ingested. The first stage is a **Native-First Inspection**. Using a fast, low-level parser like `PyMuPDF` or `pdfminer.six`, the system extracts all available text, font information, bounding boxes, and links without any rasterization ¹¹ ¹⁰³. This step is extremely fast and sufficient for high-quality, born-digital PDFs. The extracted native text is then passed to a series of **Quality Gates**. These gates are not based on vague AI confidence scores but on concrete, measurable signals. The system checks for garbled text, encoding corruption, missing glyphs, invisible text layers, and whether the text aligns visually with the underlying image of the page ⁶³. If the native text passes these gates with high confidence, it is immediately passed to the renderer. This path ensures near-instantaneous processing for simple documents.

If the native text fails any quality gate, the router triggers a conditional escalation. The problematic region is flagged for higher-fidelity processing. The router then decides the optimal next step based on the nature of the failure. For example, if reading order is disjointed, it may invoke a layout analysis model like `MinerU` to establish a plausible sequence ⁶. If the text is completely illegible, the router directs the corresponding page region to be rendered at a high resolution (e.g., 300 DPI or higher) and processed by an advanced OCR engine like `PaddleOCR-VL` ³³ ⁶⁷. This dynamic adjustment of DPI is a key optimization; low-quality scans might require 300+ DPI, while clean text might only need 72 DPI for visual alignment checks.

For specialized content, the router invokes dedicated processors. When a table is detected, it is passed to a specialized table parser, potentially using techniques from benchmarks like `OmniDocBench` to handle complex structures like spanning cells and nested tables ⁸⁷. Similarly, when a mathematical expression is identified, it is routed to a model like `Nougat`, which is specifically trained to recognize LaTeX syntax in academic documents ⁸⁶. This specialization ensures that the best tool is always used for the job, rather than forcing all content through a generic OCR pipeline.

Finally, after both native and escalated extractions are complete, a **VLM Repair Loop** acts as a final verification pass. This is a critical departure from Marker's primary reliance on VLMs. In Philon, the VLM is not the first line of defense but a fine-tuning mechanism. It receives the initial extraction and is tasked with identifying and correcting specific errors, such as misidentified symbols or broken structure, based on its visual understanding of the page ¹⁸. This selective block-level repair is far more efficient and cost-effective than regenerating entire pages ¹⁸. The router uses disagreement checks between the native, OCR, and VLM-derived results to decide whether a block needs repair and to generate a final, consensus output. This multi-layered, tiered approach ensures maximum accuracy for complex documents while providing exceptional speed for simpler ones, forming the architectural backbone of Philon.

graph TD

```

A[Start: Ingest PDF] --> B{Native-First Inspection<br>via PyMuPDF/pdfm}
B -- Passes Quality Gates --> M[Renderer: Generate Markdown/HTML]
B -- Fails Quality Gates --> C[Router: Classify Failure Type]

C -->|Garbled Text/No Native| D[High-Res Rasterize (300 DPI+)]
C -->|Disjointed Layout| E[Layout Analysis (MinerU/LayoutParser)]
C -->|Specialized Content| F[Invoke Specialized Processor<br>(Nougat f

D --> G[OCR Engine (PaddleOCR-VL)]
E --> H[Re-establish Reading Order]
F --> I[Process Specialized Content]

G --> J[Assemble IR]
H --> J
I --> J

J --> K[VLM Repair Loop<br>(Selective Block Correction)]
K --> L[Finalize Document IR]
L --> M

```

This architecture supports progressive output, cancellation, caching, and resumability by operating on a block-by-block basis. Each block's processing status and results can be cached, allowing a failed job to resume from the last successful step. The router's logic is designed to be deterministic in local-only mode, ensuring reproducible results, while still supporting an optional, pluggable cloud-assisted mode for tasks that exceed local hardware capabilities ³⁸.

Canonical Document Internal Representation (IR)

The cornerstone of Philon's architecture is its canonical internal representation (IR)—a versioned, loss-aware, and meticulously detailed JSON structure that serves as the universal intermediate format for all document data. This IR decouples the complex, multi-stage extraction process from the final rendering phase, enabling deterministic outputs, full source provenance, and extensibility to multiple export formats [2](#) [19](#). The design is heavily influenced by the success of similar structured JSON outputs from competitors like Unstructured and Docling, which have become de facto standards for transforming unstructured files into schema-ready inputs [5](#) [26](#) [94](#). The Philon IR is designed to be both human-readable for debugging and machine-processable for downstream applications like RAG pipelines [14](#).

The top-level object in the IR is the **Document** object. It contains global metadata about the source file, such as title, author, creation date, and page count, along with a **version** field (e.g., "1.1.0") to manage schema evolution. The **Document** object holds an array of **Page** objects, each representing a single page of the original document.

Each **Page** object contains the page number (1-indexed) and an array of **Block** objects. A **Block** is a polymorphic container that represents a logical unit of content on the page. It has a mandatory **type** field (e.g., 'paragraph', 'heading', 'list', 'table', 'figure', 'code_block', 'horizontal_rule') and an array of **InlineSpan** objects that represent formatted text segments within it. This separation of block-level structure from inline formatting (like bold or italic) provides granular control over rendering.

Every single piece of content within a **Block** or **InlineSpan** is accompanied by a rich **source_map** object. This is the most critical component for ensuring fidelity and trustworthiness. The **source_map** provides an immutable record of the content's origin. It includes the **pdf_page** number, a **bounding_box** array [x1, y1, x2, y2] defining the exact location in points, the **extraction_method** used (e.g., 'native_text', 'ocr_surya', 'nougat_formula'), and a numerical **confidence** score (0.0 to 1.0). Furthermore, it can contain an array of alternative **extractions** if multiple methods were tried and disagreed, allowing for sophisticated error analysis and repair. This level of detail is essential for creating verifiable golden-output test suites and for providing users with a clear understanding of the conversion's certainty [13](#) [63](#).

Specialized block types have additional properties. A **Table** block contains a two-dimensional array of **Cell** objects. Each **Cell** can hold its own **Block** content, allowing for nested tables. A **Figure** block contains a reference to an embedded asset and a **caption** field, which is itself a **Block** object, allowing captions to be semantically structured. Inline spans can also reference links and citations, with their own **source_map** pointing back to the relevant anchors in the document.

Below is a simplified schema illustrating the structure:

```
{
  "version": "1.1.0",
  "metadata": {
    "title": "Example Research Paper",
    "author": "Jane Doe",
    "page_count": 5
  },
  "pages": [
    {
      "page_number": 1,
      "blocks": [
        {
          "id": "b1",
          "type": "heading",
          "depth": 1,
          "children": [
            {
              "text": "An Introduction to Advanced Document Conversion",
              "spans": []
            }
          ]
        },
        {
          "id": "b2",
          "type": "paragraph",
```

```

    "children": [
      {
        "text": "This is a sample paragraph. The Philon platform aims to help you",
        "spans": []
      },
      {
        "text": "See the following table:",
        "spans": []
      }
    ],
    "source_map": {
      "pdf_page": 1,
      "bounding_box": [72, 680, 540, 700],
      "extraction_method": "native_text",
      "confidence": 0.95
    }
  },
  {
    "id": "b3",
    "type": "table",
    "children": [
      [
        { "text": "Header 1", "spans": [] },
        { "text": "Header 2", "spans": [] }
      ],
      [
        { "text": "Row 1, Cell 1", "spans": [] },
        { "text": "Row 1, Cell 2", "spans": [] }
      ]
    ],
    "source_map": {
      "pdf_page": 1,
      "bounding_box": [72, 600, 540, 640],
      "extraction_method": "specialized_table_parser_omnidocbench_v2",
      "confidence": 0.98
    }
  }
]
}

```

```
]
}
```

This detailed, loss-aware IR provides the necessary granularity to support all required output modes deterministically. It allows for the creation of faithful Markdown and accessible HTML renderers, and its structured nature makes it an ideal input for chunking algorithms for RAG systems. By treating the IR as a first-class citizen, Philon establishes a robust and transparent foundation for high-fidelity document conversion.

Output Strategy: Fidelity Guarantees for Markdown, HTML, and JSON/IR

Philon's output strategy will be productized in a phased, prioritized manner, starting with formats that offer the highest utility and value to the core user base. The primary goal is to deliver outputs that are not only syntactically correct but also semantically meaningful, structurally sound, and faithfully represent the source document. A key aspect of this strategy is establishing clear fidelity guarantees and documenting irreducible limitations to manage user expectations.

v1: Essential Outputs - Markdown, Semantic HTML, and JSON/IR

The initial release (v1) will focus on three core formats, all derived from the canonical Document IR.

- 1. Markdown (Optimized for Humans and RAG):** The Markdown output will be a dual-purpose format. For human consumption, it will prioritize clean, readable prose, correctly rendering headings, lists, emphasis, and blockquotes. For RAG pipelines, its primary strength will be the accurate preservation of structural elements that are often lost in naive conversions. This includes correctly formatted tables, fenced code blocks with proper language identifiers, and clear delineation of figures and other elements [55](#) [57](#). To achieve this, Philon will use a robust Markdown compiler like `marked.js` or `intellij-markdown` which are designed for speed and correctness [8](#) [9](#). Fidelity guarantees for Markdown include:
 - **Structure:** All heading levels, ordered/unordered lists, and blockquote elements will be preserved with correct nesting.

- **Tables:** Table structure (rows and columns) will be maintained using standard Markdown table syntax. Spanning cells will be handled gracefully, though complex nested tables may be simplified.
- **Code Blocks:** Text identified as code will be wrapped in triple-backtick fences with an optional language tag.
- **Irreducible Limitations:** Extremely complex, hand-drawn diagrams or custom vector graphics cannot be converted into text-based representations. Handwriting will be converted to typed text via OCR, losing the original stylistic nuances.

1. **Semantic and Accessible HTML5:** This is a key differentiator for Philon. The HTML output will not be a simple translation but a purpose-built, accessible web document. It will adhere strictly to HTML5 standards and be validated to ensure well-formedness ². Crucially, it will be designed for accessibility, conforming to WCAG 2.2 guidelines ^{79 84}. This involves using appropriate semantic tags (`<header>`, `<main>`, `<section>`, `<article>`, `<table>`, `<caption>`, `<figcaption>`), adding ARIA roles where necessary, and ensuring proper contrast ratios. To maintain the core tenet of provenance, every element in the generated HTML will include `data-*` attributes containing the `source_map` information from the IR. For example, `<p data-provenance-id="b2">...</p>` would link the paragraph back to its extraction record ⁸¹. Post-conversion, the HTML can be automatically checked for accessibility compliance using tools like `axe-core` ^{99 101}. Fidelity guarantees for HTML include:

- **Validity and Semantics:** The output will be valid, well-formed HTML5 that accurately reflects the document's structure.
- **Accessibility:** The document will be marked as compliant with WCAG 2.2 Level AA standards.
- **Asset Handling:** Images and other assets will be embedded or linked correctly.
- **Irreducible Limitations:** Complex CSS styling, interactive form fields, and certain advanced SVG features may not be perfectly replicated.

1. **JSON/Document IR:** The canonical Document IR itself will be a primary export format. This provides users with a machine-readable, structured view of the parsed document, ready for programmatic manipulation, chunking for RAG, or feeding into other data processing pipelines ^{44 94}. This output is guaranteed to be a lossless representation of the data extracted by the Philon pipeline, including all provenance information.

v2: Advanced Outputs - DOCX and EPUB

Following the stable release of v1, Philon will introduce support for widely used office and e-book formats. These will be implemented as secondary renderers built upon the canonical IR.

1. **DOCX:** This format requires careful handling of styles, fonts, and layout. The Philon DOCX renderer will translate the semantic structure from the IR into corresponding Word styles (e.g., 'Heading 1', 'Intense Quote'). It will embed images and preserve hyperlinks. Fidelity guarantees will focus on maintaining the logical structure and readability of the document. The main limitation will be the faithful replication of the exact pixel-perfect position and appearance of complex, multi-column layouts from the original PDF.
2. **EPUB:** Converting to EPUB presents challenges related to reflowable content and asset packaging ³. The Philon EPUB renderer will create a standard, accessible EPUB3 file from the semantic HTML output, ensuring that images, CSS, and JavaScript (if any) are packaged correctly. The primary guarantee will be a device-independent, accessible reading experience that adapts to different screen sizes.

v3: Niche Outputs - TEI, JATS, ALTO, hOCR

These formats cater to specific academic, archival, and historical preservation communities. Their inclusion will depend on user demand gathered post-v1 launch.

- **TEI (Text Encoding Initiative) & JATS (Journal Archiving and Interchange Tag Set):** These XML-based formats are used for scholarly publishing. Philon could generate these by mapping its `Block` types to the appropriate TEI/JATS tags.
- **ALTO & hOCR:** These are XML formats for representing OCR results and their spatial locations. They are valuable for digitization projects and would be a natural extension of Philon's commitment to provenance. The output would map Philon's `source\_map` data directly into the corresponding XML attributes.

This phased approach allows Philon to deliver immediate value with the most versatile formats (Markdown, HTML, JSON) while strategically planning for broader compatibility, ensuring that development resources are allocated efficiently.

Benchmark Blueprint for Measurable Quality and Performance

To validate Philon's architectural advantages and provide verifiable proof of its superiority, a public, reproducible, and rigorously designed benchmark is essential. This blueprint outlines the components of the benchmark, drawing on established practices and addressing the need to prevent benchmark gaming. The evaluation system will measure Philon across a wide spectrum of metrics, including text fidelity, structural accuracy, speed, and resource consumption, against a diverse set of baselines.

Test Corpus Composition

The benchmark corpus must be representative of the real-world diversity of documents Philon is expected to process. It will be composed of multiple subsets, each targeting a different dimension of difficulty.

- **Document Classes:** The corpus will include nine distinct document classes, mirroring the scope of benchmarks like OmniDocBench ⁸⁹ ⁹⁰. These will range from academic papers and textbooks to legal filings, invoices, magazines, slideshows, and scientific reports. This variety ensures the evaluation captures performance across different structural complexities.
- **Document Quality:** Subsets will be created based on scan quality, ranging from high-resolution, born-digital PDFs to low-quality, noisy scans and documents with degraded print. This is critical for assessing the effectiveness of the adaptive routing logic.
- **Language and Scripts:** The benchmark will include documents written in multiple languages, with a particular focus on challenging scripts like CJK (Chinese, Japanese, Korean) and RTL (Right-to-Left) languages like Arabic and Hebrew, for which OCR accuracy can be a significant challenge ³³.
- **Complexity and Length:** The corpus will feature documents of varying lengths, from single-page invoices to multi-hundred-page books. It will also include a specific "hard subset" focusing on documents with difficult formulas, complex tables, and dense layouts, similar to the one added to OmniDocBench ¹⁰⁶.

Gold Data and Annotation Strategy

High-quality ground truth (gold data) is the bedrock of any meaningful benchmark. For Philon, this gold data will be the canonical JSON IR produced by a meticulous manual annotation process or, for synthetic documents, derived directly from the source markup

(e.g., LaTeX for papers) 49. The annotation strategy will follow the Philon IR schema, capturing not just the text but also the intended structure (headings, lists), tables, figures, and mathematical expressions. To ensure the long-term quality and relevance of the gold data, the benchmark will adopt a community review model inspired by PureDocBench, allowing the community to suggest corrections and improvements to the ground truth annotations 109.

Metrics for Evaluation

The evaluation harness will compute a suite of quantitative metrics across multiple axes of quality.

Metric Category	Specific Metrics	Description
Text Fidelity	Character Error Rate (CER), Word Error Rate (WER)	Measures the accuracy of raw text extraction by comparing the output to the gold-standard text.
Structural Accuracy	Heading/List Structure F1-Score, Reading Order Accuracy	Evaluates the correctness of the document's hierarchy and logical flow. Reading order can be measured against datasets like ReadingBank 41.
Table & Formula Extraction	Table Cell-level Precision/Recall/F1, Formula Reconstruction Accuracy	Assesses the ability to correctly identify and extract the contents of tables and mathematical expressions. Formulas will be scored by comparing the reconstructed LaTeX to the ground-truth LaTeX 87.
HTML Validity & Accessibility	W3C Validator Score, axe-core Violation Count	Automated checks to ensure the generated HTML is well-formed and compliant with WCAG 2.2 accessibility standards 99 101.
Source Map Correctness	Provenance Hit Rate	The percentage of characters in the output that can be successfully mapped back to a unique source location in the PDF.
Speed & Performance	Cold/Warm Latency (ms/page), Throughput (pages/sec), GPU/CPU/MPS Utilization, Memory/VRAM Usage, Cost/Page (for cloud models)	Comprehensive measurement of processing time and resource consumption, separating cold starts from steady-state performance 36.

Baseline Comparison and Anti-Gaming Controls

To provide context for Philon's performance, the benchmark will compare it against a wide range of state-of-the-art solutions. This includes open-source tools like Marker, Docling, MinerU, and Unstructured, as well as leading commercial APIs like Mathpix and LlamaParse 7 35 36. Comparisons will be clearly separated into local/open-source and managed API categories to ensure a fair, apples-to-apples evaluation.

To prevent benchmark gaming, where developers might inadvertently tune their models to perform well on a static test set, several controls will be implemented. The full benchmark dataset will be kept private until evaluation periods and will be updated

regularly with new, challenging documents. The scoring will emphasize structural and semantic accuracy (e.g., table structure, heading hierarchy) over simple character matching, as the latter is easier to game. Finally, the CI regression strategy will involve running nightly tests on a small, curated subset of the benchmark to catch regressions early and ensure that new commits do not degrade existing functionality ⁴⁵.

Implementation Roadmap and Technology Stack

This section outlines a concrete 30/60/90-day implementation roadmap for building Philon, including technology choices, module boundaries, and risk mitigation strategies. The plan is designed to deliver a functional v1 product with the core adaptive routing architecture and essential output formats.

Technology Stack and Dependencies

The selection of technologies will prioritize a permissive license (MIT or Apache 2.0) to enable broad adoption and commercial use, explicitly avoiding the GPL-licensed Marker codebase ⁷⁰ ⁷².

Component	Recommended Choice	Alternatives	Justification
Language/ Runtime	Python 3.10+	Rust (with PyO3 bindings)	Mature ecosystem for ML, data science, and scripting. Fast development cycle.
PDF Parsing	PyMuPDF (fitz)	pdfminer.six	Excellent performance and comprehensive features for low-level PDF inspection. pdfminer is a strong alternative for deep structural analysis.
OCR Engine	PaddleOCR	Surya	Permissive Apache 2.0 license; excellent multilingual support and high accuracy. Surya is also a strong contender with a permissive license.
Layout Analysis	MinerU	Custom LayoutParser model	Provides pre-trained models for element detection. A custom model could be trained on OmniDocBench data for higher accuracy.
Formula Conversion	Nougat	DeepSeek-OCR	Specifically designed for academic PDFs and LaTeX conversion. High accuracy in this niche.
VLM/Repair	Open-source models from OpenCompass or Groq/OpenBench	Paid APIs (e.g., Claude, GPT-4V)	Allows for experimentation with repair capabilities before committing to paid services.
Markdown Rendering	marked.js (via Node.js bridge) or mistune	intellij-markdown (Kotlin)	Highly optimized and standards-compliant libraries for generating Markdown.
HTML Validation	html5validator	Built-in browser validators	Ensures generated HTML is well-formed and valid.
Accessibility Testing	axe-core (via Playwright/Selenium)	pally	Industry-standard tool for automated accessibility auditing.
Evaluation Framework	deepEval or custom script	OpenCompass	Modular and easy-to-use for defining evaluation tests as "unit tests" for LLM systems.

Module and File Layout Proposal

The project will be organized into a modular Python package structure:

```
philon/
├── philon/core/           # Core logic and interfaces
│   ├── router.py         # Main routing orchestration
│   ├── ir.py             # Document IR schema and serializers
│   └── job.py             # Job model for tracking state
├── philon/extractors/    # Pluggable extractor modules
│   ├── __init__.py
│   ├── native.py         # Implements native text extraction
│   ├── ocr.py            # Wraps PaddleOCR
│   ├── table.py          # Wraps specialized table parser
│   └── formula.py        # Wraps Nougat
├── philon/renderers/     # Pluggable renderer modules
│   └── __init__.py
```

		markdown.py	# Renders IR to Markdown
		html.py	# Renders IR to Semantic HTML
		philon/benchmark/	# Benchmarking harness
		corpus/	# Test datasets
		scorers.py	# Metric calculation
		harness.py	# Runs evaluation
		cli.py	# Command-line interface

30/60/90-Day Build Plan

- **Phase 1: Foundation (Days 1-30)**

- **Goals:** Establish the core infrastructure, define the IR, and implement the native-first extraction and basic rendering.

- **Tasks:**

1. Set up the Python project with Poetry for dependency management and
2. Define the canonical JSON IR schema in `pydantic` models and implement
3. Implement the `NativeTextExtractor` using `PyMuPDF`. This module will
4. Create the initial `MarkdownRenderer` that can render basic blocks
5. Curate a small, initial benchmark dataset of ~20 simple, born-digital
6. Implement basic CER and WER metrics to evaluate the native extractor

- **Milestones:** Version 0.1.0 released. Core IR and native extraction pipeline functional. Basic benchmark produces scores.

- **Phase 2: Expansion and Intelligence (Days 31-60)**

- **Goals:** Integrate the adaptive routing logic, add support for complex elements, and implement the semantic HTML output.

- **Tasks:**

1. Develop the `Router` logic. Implement quality gates to detect garbage
2. Integrate the `OcrExtractor` (PaddleOCR) and connect it to the router
3. Add a `TableDetector` and integrate a specialized table parsing library
4. Integrate the `FormulaConverter` (Nougat) to handle mathematical expressions
5. Implement the `HtmlRenderer` with support for semantic tags and `div`
6. Expand the benchmark with documents containing tables, formulas, and

- **Milestones:** Version 0.2.0 released. Adaptive routing and multi-element extraction functional. v1 output formats (MD, HTML, JSON) complete.

- **Phase 3: Refinement and Validation (Days 61-90)**

- **Goals:** Implement the VLM repair loop, add advanced output formats, and conduct a full comparative evaluation.

- **Tasks:**

1. Experiment with open-source VLMs to implement a selective block-level repair.
2. Implement the `DocxRenderer` and `EpubRenderer` based on the canonical output.
3. Conduct a full comparative benchmark against Marker, Docling, and PyMuPDF.
4. Publish the initial benchmark results and make the evaluation harness available.
5. Establish a CI/CD pipeline with nightly regression tests on the benchmark.

- **Milestones:** Version 1.0.0 alpha released. Full feature set complete. Public benchmark and results published.

Testing Plan and Risks

The testing plan will be comprehensive, covering unit, integration, golden-output, visual diffing, fuzzing, and adversarial PDF tests. Unit tests will cover individual functions. Integration tests will verify the interaction between modules. Golden-output tests will compare Philon's output against a trusted, manually-curated "golden" output for a set of key documents ¹³. Visual diffing will be used to compare rendered outputs (e.g., HTML previews) against the golden standard. Fuzzing and adversarial PDF generation will stress-test the parser against malformed or malicious documents.

Key Risks:

- **Performance:** The adaptive routing logic may prove too slow to be practical if the overhead of classification and routing outweighs the benefits of skipping OCR.
Mitigation: Profile the router heavily during Phase 2 and optimize decision logic.
- **Accuracy:** Achieving the desired accuracy with permissively licensed open-source models may be challenging, especially for niche tasks like form extraction.
Mitigation: Continuously benchmark against Marker and be prepared to integrate proprietary APIs as a premium, optional feature.
- **Licensing:** Accidentally depending on a GPL-licensed library remains a risk.
Mitigation: Use tools like `license-checker` in the CI pipeline to enforce a whitelist of approved licenses (MIT, Apache 2.0).

Open Questions for Product Decision: 1. What is the acceptable trade-off between latency and accuracy for the target user persona? Should the default behavior favor speed or maximum fidelity? 2. Should the VLM repair loop be included in v1, or delayed to a later version to reduce complexity? 3. What is the initial target market? Is it primarily

academic/research (favoring LaTeX and tables) or enterprise/business (favoring forms and invoices)?

Final Decision Log

- **Recommended Technologies:**

1. `<strong>Python:</strong>` For its vast ecosystem and rapid development.
2. `<strong>`PyMuPDF` + `pdfminer.six`:</strong>` For fast, low-level native PDF processing.
3. `<strong>`PaddleOCR`:</strong>` For a powerful, permissively licensed OCR engine.
4. `<strong>`MinerU`:</strong>` As a starting point for layout analysis.
5. `<strong>`Nougat`:</strong>` For specialized, high-accuracy formula conversion.
6. `<strong>`marked.js` / `intellij-markdown`:</strong>` For robust Markdown rendering.
7. `<strong>`axe-core`:</strong>` For automated accessibility validation.
8. `<strong>`deepEval`:</strong>` For a flexible evaluation framework.

- **Do Not Use / Do Not Depend On:**

1. `<strong>Marker Codebase:</strong>` Due to the restrictive GPL license,
 2. `<strong>Tesseract OCR:</strong>` While permissive, it is demonstrably out of date.
 3. `<strong>Proprietary APIs (as defaults):</strong>` While they offer high accuracy,
-

Reference

1. Evaluation Metric for Document Key Information Extraction <https://arxiv.org/html/2503.05488v2>
2. From Static to Dynamic: Converting PDF Documents to HTML with Python <https://medium.com/@sirio1234/from-static-to-dynamic-converting-pdf-documents-to-html-with-python-aa2772ac0fba>
3. PDF Conversion SDK - Convert PDFs to DOCX, XLSX, Images - Avanquest <https://developers.avanquest.com/features-sdk/conversion-sdk>
4. Auto-tagging and conversion to AI-friendly formats - Nutrient <https://www.nutrient.io/blog/auto-tagging-and-document-accessibility-in-dotnet-sdk/>
5. Docling vs Unstructured vs Marker vs Visual Pipeline <https://www.ertas.ai/blog/pdf-parsing-accuracy-benchmark-docling-unstructured>

6. Docling vs Marker vs MinerU: The Ultimate Open-Source PDF ... <https://adityamangal98.medium.com/docling-vs-marker-vs-mineru-the-ultimate-open-source-pdf-parser-benchmark-2026-which-is-best-a36ecbb6c6b1>
7. The Real Open-Source PDF Parser Benchmark (2026) <https://www.youtube.com/watch?v=8RxT5jTcemY>
8. markedjs/marked: A markdown parser and compiler. Built ... <https://github.com/markedjs/marked>
9. JetBrains/markdown: Markdown parser written in kotlin <https://github.com/JetBrains/markdown>
10. Stirling PDF - The Open-Source PDF Platform <https://github.com/Stirling-Tools/stirling-pdf>
11. yzlabai/docparse-rs: Pure-Rust document parsing ... <https://github.com/yzlabai/docparse-rs>
12. Working with non-code files <https://docs.github.com/en/repositories/working-with-files/using-files/working-with-non-code-files>
13. jakewvincent/mkdnflow.nvim: Fluent navigation and ... <https://github.com/jakewvincent/mkdnflow.nvim>
14. my-stars/README.md at master <https://github.com/kissgyorgy/my-stars/blob/master/README.md>
15. For future reference but maybe not. <https://gist.github.com/tkersey/e4d9923922d80c065f9d>
16. GitHub - punkpeye/awesome-mcp-servers: A collection of ... <https://github.com/punkpeye/awesome-mcp-servers>
17. Yigtwxx/awesome-rag-production <https://github.com/Yigtwxx/awesome-rag-production>
18. Introducing: Extract <https://unstructured.io/blog/introducing-extract>
19. Structured vs. Unstructured Data: Differences and ETL Issues <https://unstructured.io/insights/structured-vs-unstructured-data-5-transformation-methods>
20. What Is Unstructured Data? Enterprise Guide to AI <https://unstructured.io/insights/understanding-unstructured-data-for-enterprise-ai>
21. Unstructured Data Platform for GenAI | Unstructured <https://unstructured.io/>
22. Web Scraping for LLMs: Chunking by Document Elements <https://unstructured.io/blog/easy-web-scraping-and-chunking-by-document-elements-for-llms>
23. What Is Data Ingestion? Foundations of Data Pipelines <https://unstructured.io/insights/data-ingestion-building-modern-data-pipelines>

24. What Matters for LLM Data Ingestion and Preprocessing <https://unstructured.io/blog/understanding-what-matters-for-llm-ingestion-and-preprocessing>
25. End-to-End Data Workflow Integration: Source to Use <https://unstructured.io/insights/end-to-end-data-workflows-from-raw-files-to-ai-applications>
26. Unstructured + Databricks Delta Tables: RAG Quickstart <https://unstructured.io/blog/getting-started-with-unstructured-and-delta-tables-in-databricks>
27. Preparing Unstructured Data for Analytics and AI Systems <https://unstructured.io/insights/unstructured-data-preparation-the-complete-ai-pipeline-guide>
28. Which Small vLLM OCR Model Is the Best For Private Use ... <https://medium.com/@AiDocTakes/i-tested-four-ocr-models-on-scanned-medical-records-and-the-smallest-one-won-ed7185b1c0b2>
29. Best Open Source OCR Tools 2026: Tesseract, EasyOCR, ... <https://imagetotable.ai/blog/best-open-source-ocr-tools-2026>
30. Best Open Source OCR Tools & Models in 2026 <https://unstract.com/blog/best-opensource-ocr-tools/>
31. Top 20 OCR AI Models 2026: Local Extraction <https://local-ai-zone.github.io/guides/best-ai-ocr-models-ultimate-ranking-2026.html>
32. Best Open Source OCR for AI Agents (2026 Complete Guide) <https://www.madebyagents.com/blog/best-open-source-ocr-for-ai-agents>
33. Best OCR Models in 2026 (Updated April 2026) <https://gigagpu.com/best-ocr-models-2026/>
34. Best Open Source PDF to Markdown Tools (2026): Marker vs ... <https://jimmysong.io/blog/pdf-to-markdown-open-source-deep-dive/>
35. PDF to Markdown converter benchmark: 9 tools — Docling ... <https://github.com/pdfmarkdownapp/pdf-to-markdown-benchmark>
36. PDF Data Extraction Benchmark 2025: Comparing Docling ... <https://procycons.com/en/blogs/pdf-data-extraction-benchmark/>
37. datalab-to/marker: Convert PDF to markdown + JSON ... <https://github.com/datalab-to/marker>
38. Best PDF to Markdown Converter in 2026: Ranked for AI and ... <https://www.file2markdown.ai/blog/best-pdf-to-markdown-converter>
39. marker-pdf 0.3.2 <https://pypi.org/project/marker-pdf/0.3.2/>
40. DocBank: A Benchmark Dataset for Document Layout ... <https://github.com/doc-analysis/DocBank>
41. ReadingBank: A Benchmark Dataset for Reading Order ... <https://github.com/doc-analysis/ReadingBank>

42. Top Python Libraries of 2025 <https://www.edge-ai-vision.com/2026/01/top-python-libraries-of-2025/>
43. Awesome Open Source AI <https://github.com/alvinreal/awesome-opensource-ai>
44. What Is Information Retrieval for AI Applications? <https://unstructured.io/insights/what-is-information-retrieval-for-ai-applications>
45. finos-labs/ai-evals-framework <https://github.com/finos-labs/ai-evals-framework>
46. GSoC 2026 – Interest in Behavioral Evaluation Test ... <https://github.com/google-gemini/gemini-cli/discussions/20482>
47. GSoC 2026: Behavioral Eval Framework <https://github.com/google-gemini/gemini-cli/discussions/19604>
48. confident-ai/deepeval: The LLM Evaluation Framework <https://github.com/confident-ai/deepeval>
49. Benchmarking PDF Parsers on Table Extraction with LLM-based ... - arXiv <https://arxiv.org/html/2603.18652v1>
50. Benchmarking Vision-Language Models for French PDF-to ... <https://www.semanticscholar.org/paper/985050b4db8d1b99d337d7c8137393581e1337c3>
51. OmniDocBench: Benchmarking Diverse PDF Document Parsing ... <https://cvpr.thecvf.com/virtual/2025/poster/34400>
52. Semantic HTML in 2025: The Bedrock of Accessible, SEO-Ready, and ... <https://dev.to/gerryleonugroho/semantic-html-in-2025-the-bedrock-of-accessible-seo-ready-and-future-proof-web-experiences-2k01>
53. Benchmarking Vision-Language Models for French PDF-to ... https://www.researchgate.net/publication/400742819_Benchmarking_Vision-Language_Models_for_French_PDF-to-Markdown_Conversion
54. \readoc: A Unified Benchmark for Realistic Document ... <https://arxiv.org/html/2409.05137v1>
55. Best PDF to Markdown Converters for RAG in 2026 <https://www.searchcans.com/blog/pdf-markdown-rag-tools/>
56. Evaluating Your Parsing Solution: A Comparative Look at ... <https://www.cambioml.com/en/blog/evaluate-document-parsing-accuracy>
57. PDF to Markdown Converter — LLM-Ready for RAG <https://tools.nanonets.com/pdf-to-markdown>
58. Unstructured Evaluation Metrics (SCORE Framework) <https://github.com/Unstructured-IO/unstructured-eval-metrics>
59. OpenCompass is an LLM evaluation platform, supporting a ... <https://github.com/open-compass/opencompass>

60. GitHub - groq/openbench: Provider-agnostic, open-source ... <https://github.com/groq/openbench>
61. Top 5 Open-Source LLM Evaluation Frameworks in 2026 <https://dev.to/guybuildingai/-top-5-open-source-llm-evaluation-frameworks-in-2024-98m>
62. PDF to Markdown in 2026: The Techniques That Actually ... <https://nocodeapi.com/tutorials/pdf-to-markdown-in-2026-the-techniques-that-actually-work/>
63. PDF to Markdown: Best Tools, Comparison, Limitations ... <https://unstract.com/blog/why-pdf-to-markdown-ocr-fails-for-ai-document-processing/>
64. Accurate PDF to Markdown Conversion at 87x Faster Speed https://www.linkedin.com/posts/jonathan-rhyne-54084811_most-pdf-extractors-make-you-choose-fast-activity-745280183536044416-VfPw
65. OCR SOTA Router 2026: Choose OCR by Output Contract <https://www.codesota.com/ocr>
66. What is the best Open Source OCR in 2026? : r/LocalLLaMA https://www.reddit.com/r/LocalLLaMA/comments/1sk6kst/what_is_the_best_open_source_ocr_in_2026/
67. 7 Best Open-Source OCR Models 2025: Benchmarks & ... <https://www.e2enetworks.com/blog/complete-guide-open-source-ocr-models-2025>
68. Curated list of open-Source OCR models and toolkit https://www.linkedin.com/posts/aboniasojasingarayar_ocr-opensource-documentai-activity-7391736028996313088-gLfu
69. adithya-s-k/marker-api - GitHub <https://github.com/adithya-s-k/marker-api>
70. How to Choose and Add a License on GitHub [2026 Guide] - YouTube <https://www.youtube.com/watch?v=AgHWSfwmfNQ&vl=en>
71. MIT License - apcamargo/pdf-converter - GitHub <https://github.com/apcamargo/pdf-converter/blob/main/LICENSE>
72. git - How to change the license for a project at Github? - Stack Overflow <https://stackoverflow.com/questions/20243214/how-to-change-the-license-for-a-project-at-github>
73. I benchmarked 4 Python text extraction libraries so you don ... https://www.reddit.com/r/Python/comments/1ls6hj5/i_benchmarked_4_python_text_extraction_libraries/
74. Marker vs Docling vs MinerU vs pdf-craft vs PyMuPDF4LLM <https://themenonlab.blog/blog/best-open-source-pdf-to-markdown-tools-2026>
75. Best (non-paid) way to turn complex PDFs into markdown https://www.reddit.com/r/LangChain/comments/1ilnftx/best_nonpaid_way_to_turn_complex_pdfs_into/
76. Best PDF to Markdown Tools 2026: Ranked by Accuracy <https://blazedocs.io/blog/best-pdf-to-markdown-tools-2026>

77. Marker: Convert PDF to Markdown quickly with high accuracy <https://news.ycombinator.com/item?id=38482007>
78. Standalone CLI wrapper and docs for Nutrient's PDF-to-Markdown extractor <https://github.com/PSPDFKit/pdf-to-markdown>
79. Web Content Accessibility Guidelines (WCAG) 2.2 - W3C <https://www.w3.org/TR/WCAG22/>
80. PDF to WCAG 2.2 Compliant HTML Conversion <https://www.math.purdue.edu/~msunkula/Accessibility/notes2html.html>
81. Creating Accessible and Responsive Web Content - March 2026 <https://digitalaccessibility.virginia.edu/converting-documents-html-creating-accessible-and-responsive-web-content-march-2026>
82. PDF to WCAG Compliant HTML and PDF Converter - AI-Powered ... <https://playground.halfaccessible.com/pdf-to-html-converter>
83. Accessible documents in 2026: WCAG 2.2, EAA enforcement, and ADA ... <https://www.nutrient.io/blog/wcag2-accessibility-requirements-documents/>
84. Understanding WCAG 2.2: The Complete Practical Guide (2026) <https://www.webability.io/blog/understanding-wcag-2-2-a-practical-guide>
85. Docling is a new library from IBM that efficiently parses ... https://www.reddit.com/r/LocalLLaMA/comments/1ghbmoq/docling_is_a_new_library_from_ibm_that/
86. Nougat: Neural Optical Understanding for Academic ... <https://github.com/facebookresearch/nougat>
87. OmniDocBench Leaderboard: PDF Document Parsing SOTA <https://codesota.com/browse/computer-vision/document-parsing/omnidocbench>
88. OmniDocBench: Benchmarking Diverse PDF Document ... <https://arxiv.org/html/2412.07626v1>
89. OmniDocBench: Benchmarking Diverse PDF Document ... https://openaccess.thecvf.com/content/CVPR2025/papers/Ouyang_OmniDocBench_Benchmarking_Diverse_PDF_Document_Parsing_with_Comprehensive_Annotations_CVPR_2025_paper.pdf?utm_source=chatgpt.com
90. OmniDocBench: Benchmarking Diverse PDF Document ... <https://arxiv.org/html/2412.07626v2>
91. OmniDocBench is Saturated, What's Next for OCR ... <https://www.llamaindex.ai/blog/omnidocbench-is-saturated-what-s-next-for-ocr-benchmarks>
92. OmniDocBench: Benchmarking Diverse PDF Document ... <https://arxiv.org/abs/2412.07626>
93. Document elements and metadata <https://docs.unstructured.io/api-reference/legacy-api/partition/document-elements>

94. Transform a JSON file into a different schema <https://docs.unstructured.io/api-reference/legacy-api/partition/transform-schemas>
95. Overview <https://docs.unstructured.io/open-source/introduction/overview>
96. Unstructured Pipelines walkthrough <https://docs.unstructured.io/pipelines/walkthrough>
97. OmniDocBench: Benchmarking Diverse PDF Document ... https://www.researchgate.net/publication/394650843_OmniDocBench_Benchmarking_Diverse_PDF_Document_Parsing_with_Comprensive_Annotations
98. OmniDocBench Leaderboard & Scores — August 2026 <https://benchlm.ai/benchmarks/omnidocbench>
99. dequelabs/axe-core: Accessibility engine for automated Web UI testing <https://github.com/dequelabs/axe-core>
100. Web Accessibility Evaluation Tools List - W3C <https://www.w3.org/WAI/test-evaluate/tools/list/>
101. What tools are you using to ensure your website is meeting ... - Reddit https://www.reddit.com/r/webdev/comments/j5b931/what_tools_are_you_using_to_ensure_your_website/
102. Datalab: Document AI Foundation Models - IDP-Software <https://idp-software.com/vendors/datalab/>
103. RAG/LLM and PDF: Conversion to Markdown Text with PyMuPDF <https://medium.com/@pymupdf/rag-llm-and-pdf-conversion-to-markdown-text-with-pymupdf-03af00259b5d>
104. Benchmarking PDF to Markdown Document Converters <https://ai.gopubby.com/benchmarking-pdf-to-markdown-document-converters-part-2-0439867a3676>
105. I Spent May Evaluating Different Engines for OCR <https://towardsdatascience.com/i-spent-may-evaluating-different-engines-for-ocr/>
106. opendatalab/OmniDocBench · Datasets at Hugging Face <https://huggingface.co/datasets/opendatalab/OmniDocBench>
107. opendatalab/MinerU: Transforms complex documents like ... <https://github.com/opendatalab/mineru>
108. Releases · opendatalab/MinerU <https://github.com/opendatalab/MinerU/releases>
109. PureDocBench: source-traceable benchmark for document ... <https://github.com/zhihengli-casia/puredocbench>
110. Create and verify PDF accessibility, Acrobat Pro <https://helpx.adobe.com/acrobat/using/create-verify-pdf-accessibility.html>

111. 10 Best Accessibility Testing Tools for WCAG 2.2 (2026) <https://crosscheck.cloud/blogs/best-accessibility-testing-tools-wcag/>